Stytch Device Fingerprinting (DFP) — Client-Side Red Team

Executive write-up · 2026-10-25 · authorized, own-stack only · no customer / prod traffic touched

> **One line:** A Stytch (Twilio) DFP SDK ships its device-intelligence telemetry encrypted with AES-256-CBC under a **per-session key that the client itself stores XOR-obfuscated in its own heap** — we recovered that key, decrypted real payloads, read the full browser-tell schema, and built a working **tamper-and-reseal** proof of concept. The per-request nonce blocks verbatim replay but not live tampering; the real boundary has to be server-side.

What we accomplished

1. **Reversed the SDK's protection end-to-end.** The public `https://elements.stytch.com/telemetry.js` is C++/Emscripten compiled to **wasm2js** (no separate `.wasm`). We mapped its full pipeline: signal collection → serialization → **AES-256-CBC** → base64 POST to `/submit`.
2. **Recovered the exact crypto and wire format.** `wire blob = [ 64B session material ][ 16B nonce = AES-256-CBC IV (plaintext) ][ AES-256-CBC ct ] AES-256 key = HEAPU8[C+96..C+128] XOR HEAPU8[8304..8336] where C = u32(HEAPU8[17349*4]) plaintext = magic(8e3421f0) | 64B session block | serialized telemetry | PKCS#7`
3. **Decrypted real payloads** — verified on many runs by the `8e3421f0` magic.
4. **Recovered the readable payload.** Via the module's own `dd()` string decoder (301 decoded field names) and its `sa()` serializer hook, we dumped the actual shipped fields: `fp_*` hashes (audio / canvas / fonts / webgl / gpu), `webdriver`, `webgl`, `UA`, `brand`, `x_*` UA-CH, screen, timezone, and bot/tamper signals.
5. **Built a working tamper-and-reseal PoC** — intercept the POST, read the key from the heap, decrypt, modify the plaintext, re-encrypt with a **fresh** nonce, reship. Server-side re-decrypt confirms the tampered payload arrives intact (`magic 8e3421f0 · payload[0:16] = "TAMPERED_PAYLOAD"`).

How we did it (methodology)

- **Runtime instrumentation (Playwright + `addInitScript`):** hooked `SubtleCrypto`, `fetch`, `getRandomValues` (to force determinism for differential analysis), the Emscripten string bridge, and the emval bridge — capturing the full ordered input stream (97 events) and heap snapshots at key moments.
- **Static RE of the wasm2js bundle:** located the AES S-box/Rcon tables, the `de()` key-schedule, the `dd()` string decoder, the `sa()` serializer, the `V()` constant decoder, and the base64 → fetch path.
- **Differential cryptoanalysis:** with randomness forced, blobs became deterministic — isolating the 16-byte plaintext nonce at header offset 64 and the true ciphertext start (offset **80**, not 112).

- **Key recovery:** the module stores its own AES key XOR-masked against a 256-byte table at heap offset 8304. `C+96 XOR table8304[0..31]` yields the real key — confirmed by successful decryption.
- **Schema recovery:** hooked `dd()` (constants → names) and `sa()` (name → value), giving the readable payload without parsing the custom wire record format.

What it proves

- The **client holds everything needed to read and rewrite its own telemetry**: key material, nonce, and format are all recoverable from the page's own memory and network — there is no server secret in the client.
- The **per-request nonce only blocks verbatim replay**. It does **not** block live tamper-and-reseal, because the nonce isn't secret and the key is recoverable per session.
- Device-intelligence signals shipped by DFP SDKs are **spoofable at the client** by a determined actor: `webdriver`, fingerprint hashes, UA, and device identity can all be altered before the server sees them.

How this could be (ab)used

Defensive framing — the same capability an attacker would use:

1. **Evading device reputation** — rewrite `webdriver → false`, change `fp_webgl` / `fp_canvas` / `fp_gpu` hashes, spoof `UA` so the fingerprint no longer matches the flagged device.
2. **Device-identity minting** — present a fresh device per request to fragment device-history correlation (same capability demonstrated on the Signifyd `device_id` in the sibling repo).
3. **Payload laundering / replay** — capture a "clean" payload and re-seal it under fresh nonces to launder reputation; combine with fresh IP/TLS to look like a new, clean device.
4. **Telemetry suppression** — decrypt, blank specific tell fields, re-encrypt, ship; the server can't tell the client lied without independent signals.

The durable boundary (what actually defends)

- **Server-side corroboration:** TLS/JA3/IP/header consistency against the claimed payload; a claim of `webdriver=false` from a known datacenter IP is already suspicious.
- **Server-issued, per-request state:** challenge nonces / HMACs over server-known data that the client must incorporate — kills pure replay and makes re-seal require solving the challenge each time.
- **Device-history + behavioral signals:** velocity, account history, and interaction patterns that can't be fully forged client-side.
- **Never base a risk verdict solely on client-supplied fingerprints** — treat them as corroboration, not truth.

Honesty / scope notes

- Authorized red-team on the user's own stacks; **public SDK only**, dummy token (`public-token-test`), local submit URLs — **no telemetry ever reached Stytch**.
- The key is **per-session** (recovered from that run's own memory), **not** a static server secret; replay is genuinely blocked by the fresh nonce. We are **not** claiming a specific fraud verdict can be forged — we prove the *client-side* payload is readable and rewritable.

See it live — screenshot of the full payload

Stytch Device Fingerprinting — decrypted, readable payload

real telemetry.js (patched with read-only sa()/dd() hooks) · SDK posts to LOCAL /submit (intercepted, never forwarded)

```
[277480] [*] SDK loaded · auto-running collection …
[277485] [*] telemetryjs() -> resolving module (C++/Emscripten wasm2js) …
[277518] [+] module resolved; awaiting Ready …
[277649] [*] GetTID() -> SDK collects signals and POSTs blob to local /submit
[277670] [+] GetTID returned; 72 signal records captured; POST intercepted (not forwarded)
[279232] [+] decrypted real blob: magic 8e3421f0 (8e3421f0)
[279734]     blob 1296B · payload 1135B · nonce 5b013cc3f94e8bcba843c50c8ef0d833
[279736]     AES-256 key = 1214f91f563f6fd900c211361be292fbf74932b2c12025ff3de8ff228cf5205f
[279738] [*] rendering 49 readable records + full decrypted wire bytes …
```

WIRE FORMAT — what actually ships (decrypted)

magic 8e3421f0 · **blob** 1296B · **payload** 1135B · **session block** 64B

nonce = AES-256-CBC IV (plaintext) 5b013cc3f94e8bcba843c50c8ef0d833

session material[4:68]

6382d74a3ae0849b9df679a4e6ec1840e2fa0c619980b97480d6d5446db55e9d89bc40341aaa6cecf42f2b835cbcb83583e25a33373f1e95147a474067a6b7bc

decrypted serialized payload (full hex)

7270887832d3995a3e1a5aa87110772cb5d85859b85d1f66e31f547d2c932f901e3cb837d897183189e94a35d1f868ff589e98b595a18f7c1b411d17913a098d350948974c4
e4620675dc916285e826bf5e7eff787257da5667fdf7a359ae17ec1164e060f59e194c2fa734f4a3c35f69fb1d3d2ec97cc860d3b30009c49318906e4409aee4632e6cf199d
4f83a82ea6158fe1b0577c136831d2d81e15ea5818e1f8b4a4c422d232e839c2dbf41a4fdfcf7550558a8986535f12c159efe9eb586775405846a3b7993302d16394907a866
70f94824899450ebccfd0b1740d8a63b5b08f967fa5c818dde941177b16bf28bf0acc45ddb82b82fd7e6884720b2612e17319ca5f8c4dff4fd84d097b5f9b5a22b57aaa829d
ec8dd78075077f58be2b7a0c2e997a802d5dd0de09e9533e58adb7f0b0577e5af12b0e54a809f6dc996e06de174f2f364d1848852d976350b3041d73291057f8840eea03ed7
fc3f80349937b4b25a7af17687895ef21be5bb7b293b51b6bbb86a48a3fb05220f50b96cf60a2ca16ae4761fc765e2970f8384ceee95d03fcd34b9fb54a867044240b46993d
38fe10bc181c41909ebfbb6c67c5963b58382aef000fe924e1c223d3565aa14b134de9c9b08f98b56b317d782d39ef7dceb558999afc6c7e33b652d09de5622a61d42383a7d
c99c4fc0b69bce106752062391bb337e6296f56f3683cf2ea0b4ecbbed30de00557f87e1e15d55d43908eeba58d7aef91520141fd9ff84b3980b15ae7894edf4054e9c91374
f4f6704052e9eedd66e7425cd59b58dc1a910478bbfe362935073d2dc128f25e5d234ae72faee45d81b9b654381d5205d489712f9521250f0907cdaef7db8792440ad6ef06d
7e9b53c9eb8fb01d794a6ac251daf50b1ed023c1f31d57ebf3b2b50119d1d3c1b7c3526b3ee12d5328efef3413f691552d97443542e251c63b07c2a7f1f88c8a5591ce16d42
077bfc9e236ea64af1b8ed7fd452c9bb47d267dfc7420025b019f23bbc2d628b0062796db2c801cd0ea740884c57d829d3223b0b5308bb5383109aa000ff3432457d477853f
9203f00c13a3d1b8b932fe78c5b33b2782dfbe000e2666b4a4583fc90b578287f55dd7decbc32134175b189936f30380e9dc75a6853bf37924dfc342600a438dfc8813218db
9126b6237802e32c3b6bf897d349d04e971bb0491bb25671486485276aaabb666d5960d99279c097d5dbb1372da9d0ada6761858b320b3256826c5379d73f30b86223c49e40
66a7c3c989a21d129030d393b0668423a1d9a51d259fd38683cfbf35f12a75807d06090644ddab92014389276c27d1f29c2d8b6c54e1e9a1832298d4f9b67f430f1471b4165
68aedff6e9b71e98ce1cc45b881c98c4a969d85a1c5876c9bf4d662072a6b5efb6bde8092f48027abcb6e942dbc8015d3610b612332f903fffc3be2a684ed50ae3633d46f02
8f010705017c321219737f952e30e3ba6fa2107f95901a186c3456b8c0b081b4004daf9cc643570451649d4cae7200ebdb9e80a8cbf2a422453099c0b2e7c29f469c6c028af
6bb568d3e5836cbade1b797732ed4f53e80678c5b8f69e

READABLE PAYLOAD — 49 fields (captured via the SDK's own sa() serializer)

FINGERPRINT HASHES (/fp_/) (9)

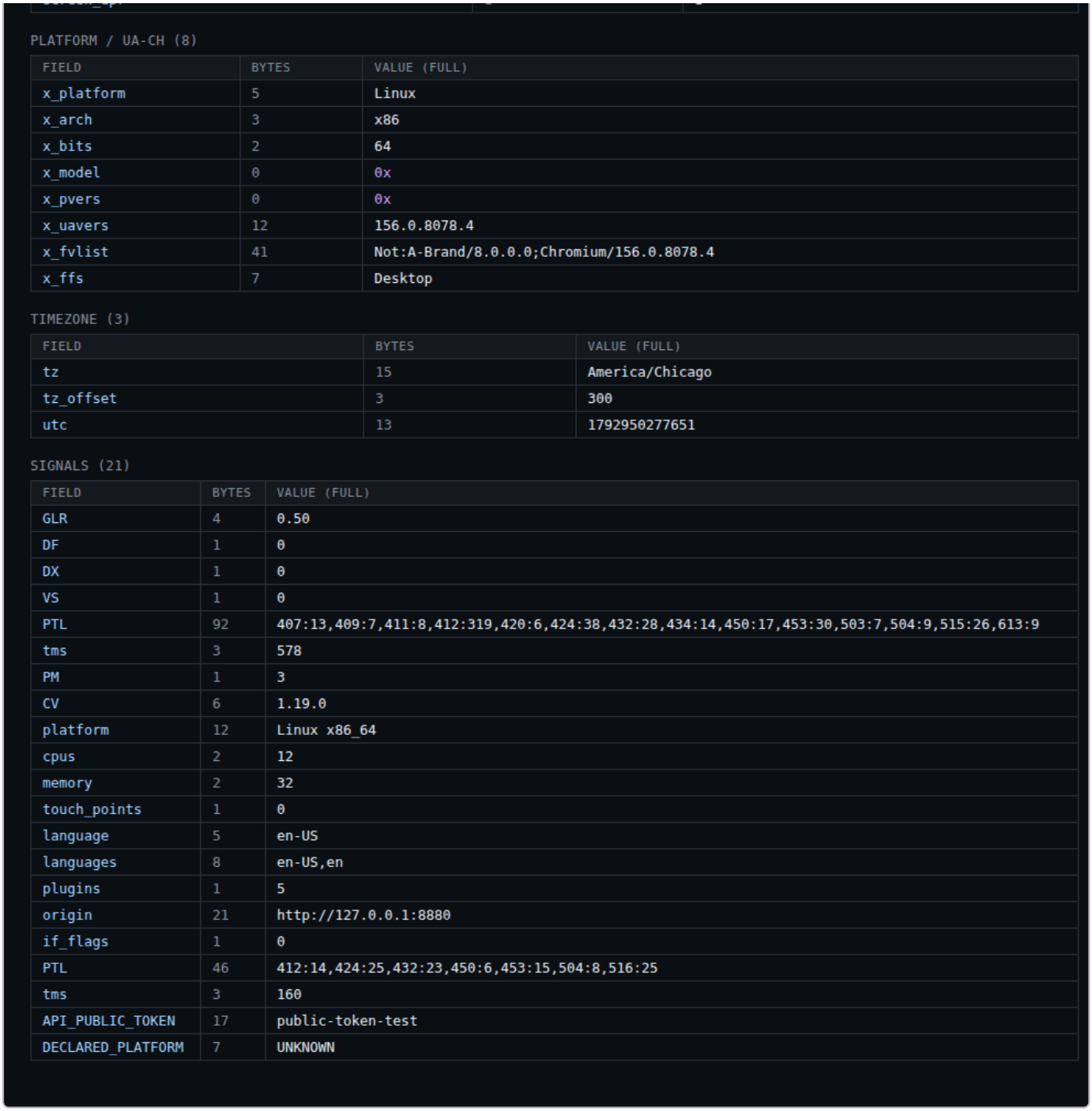
FIELD	BYTES	VALUE (FULL)
fp_audio	17	956.3163724894242
fp_feats	32	6A6033FEABFE9A6C3D74BE5B3C82B2B0
fp_perf	18	0.0999999046325684
fp_feats2	32	58E85B3373BD7DF0F3E0AFC92E74061F
fp_canvas	10	1174736476
fp_fonts	10	-979314523
fp_plugins	9	836616600
fp_webgl	11	-1057794494
fp_gpu	32	D8E2581044C03BB258F552F63A9136D2

BOT / HEADLESS TELLS (4)

FIELD	BYTES	VALUE (FULL)
brand	18	Chromium/156/Linux
webgl	115	Google Inc. (Google) / ANGLE (Google, Vulkan 1.3.0 (SwiftShader Device (Subzero) (0x0000C0DE)), SwiftShader driver)
UA	101	Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/156.0.0.0 Safari/537.36
webdriver	4	true

SCREEN (4)

FIELD	BYTES	VALUE (FULL)
screen_width	4	1000
screen_height	3	900
screen_depth	2	24
screen_dpr	1	1



PLATFORM / UA-CH (8)

FIELD	BYTES	VALUE (FULL)
x_platform	5	Linux
x_arch	3	x86
x_bits	2	64
x_model	0	0x
x_pvers	0	0x
x_uavers	12	156.0.8078.4
x_fvlist	41	Not:A-Brand/8.0.0.0;Chromium/156.0.8078.4
x_ffs	7	Desktop

TIMEZONE (3)

FIELD	BYTES	VALUE (FULL)
tz	15	America/Chicago
tz_offset	3	300
utc	13	1792950277651

SIGNALS (21)

FIELD	BYTES	VALUE (FULL)
GLR	4	0.50
DF	1	0
DX	1	0
VS	1	0
PTL	92	407:13,409:7,411:8,412:319,420:6,424:38,432:28,434:14,450:17,453:30,503:7,504:9,515:26,613:9
tms	3	578
PM	1	3
CV	6	1.19.0
platform	12	Linux x86_64
cpus	2	12
memory	2	32
touch_points	1	0
language	5	en-US
languages	8	en-US,en
plugins	1	5
origin	21	http://127.0.0.1:8880
if_flags	1	0
PTL	46	412:14,424:25,432:23,450:6,453:15,504:8,516:25
tms	3	160
API_PUBLIC_TOKEN	17	public-token-test
DECLARED_PLATFORM	7	UNKNOWN

Full-page capture of `scripts/stytch-dfp-payloadview-visual.mjs` (visible-browser demo): the decrypted wire format (magic `8e3421f0` · 64B session · plaintext nonce · full serialized hex) followed by all 49 readable payload records with the bot/headless tells highlighted. A fresh `1280B → AES-256-CBC → decrypt` run every time.

What each field means

Field	Meaning
Fingerprint hashes (`fp_*`)	
`fp_audio`	Audio-context fingerprint hash (OfflineAudioContext sample hashing)
`fp_feats` / `fp_feats2`	Browser-feature hashes (32-hex)
`fp_perf`	Performance-timing derived value
`fp_canvas`	Canvas 2D fingerprint hash

Field	Meaning
`fp_fonts`	Font-detection fingerprint hash
`fp_plugins`	Plugins fingerprint
`fp_webgl`	WebGL fingerprint hash
`fp_gpu`	GPU / WebGL-renderer hash (32-hex)
Bot / headless tells	
`webdriver`	`navigator.webdriver` — `true` = automated browser ⚠️
`brand`	UA brand string, e.g. `Chromium/156/Linux`, `HeadlessChrome/…` ⚠️
`webgl`	WebGL renderer string — `SwiftShader` = software/headless GL ⚠️
`UA`	Full `navigator.userAgent` ⚠️
Screen	
`screen_width/height`	`screen.width/height`
`screen_depth`	`screen.colorDepth`
`screen_dpr`	`window.devicePixelRatio`
Platform / UA-CH (`x_*`)	
`x_platform`	`navigator.platform` (`Linux x86_64`)
`x_arch`	UA-CH `architecture` (`x86`)
`x_bits`	UA-CH `bitness` (`64`)
`x_model` / `x_pvers`	UA-CH `model` / `platformVersion` (empty on desktop)
`x_uavers`	UA-CH `uaFullVersion` (`156.0.8078.4`)
`x_fvlist`	UA-CH `fullVersionList` brands
`x_ffs`	UA-CH `formFactors` (`Desktop`)
Timezone	

Field	Meaning
`tz` / `tz_offset` / `utc`	IANA zone, offset in minutes, epoch-ms timestamp
Signals	
`GLR`	GL renderer version
`DF` / `DX` / `VS`	Boolean flag signals (observed `0`; exact meaning not decoded)
`PTL`	Page-timing profile, `eventId:duration` pairs (two rounds)
`tms`	Timing in ms (two rounds)
`PM`	Page metric (observed `3`; not decoded)
`CV`	SDK client version (`1.19.0`)
`platform`	`navigator.platform` string
`cpus`	`navigator.hardwareConcurrency`
`memory`	`navigator.deviceMemory` (GB)
`touch_points`	`navigator.maxTouchPoints`
`language` / `languages`	Locale(s)
`plugins`	`navigator.plugins.length`
`origin`	Page origin
`if_flags`	Iframe-related flags (observed `0`; not decoded)
`API_PUBLIC_TOKEN` / `DECLARED_PLATFORM`	Config passed to `GetTID` in the demo

Marked ⚠️ = the classic headless/bot tells a fraud engine keys on — and the exact records the tamper PoC can flip before re-sealing with a fresh nonce. `DF/DX/VS/PM/if_flags` values are shown as-is; their exact semantics were not decoded.

Repo layout & quick run

```
STYTCH_DFP_RECON_2026_10_25.md   the full technical writeup (§9g decrypt,
                                 §9h tamper PoC, §9i readable payload, §10 limits)
scripts/                         all stytch-dfp-*.mjs + stytch_dfp_decrypt.py +
                                 fetch/patch scripts for the real bundle
artifacts/                       curated outputs (payloadview.json, decrypted
                                 body, keys, tamper/vis blobs) + MANIFEST.md
```

```
npm install                     # playwright (browser binary already cached)
bash scripts/fetch_and_patch_stytch.sh && python3 scripts/patch_stytch_bundle.py
node scripts/stytch-dfp-final.mjs        # capture key material + blob
python3 scripts/stytch_dfp_decrypt.py    # decrypt (magic 8e3421f0)
node scripts/stytch-dfp-payloadview.mjs  # readable payload
node scripts/stytch-dfp-tamper.mjs --dump# tamper-and-reseal, print payload
```